

AWS SDKs

Acesso Programático aos Serviços AWS

AWS Certified Developer Associate

Instrutor: Marlon Anesi

O que são AWS SDKs?

Software Development Kits - bibliotecas para interagir com serviços AWS

Principais características:

- Abstrai chamadas HTTP para APIs da AWS
- Gerenciamento automático de credenciais
- Retry automático com exponential backoff
- Paginação automática para grandes resultados
- Tipagem e autocompletar na IDE

SDKs disponíveis:

Python (boto3)

JavaScript

Java

.NET

Go

Ruby

PHP

C++

Boto3 - AWS SDK para Python

SDK mais popular, usado no exame DVA-C02!

Instalação:

```
pip install boto3
```

Conceitos-chave:

```
Client = baixo nível  
Resource = alto nível
```

```
import boto3  
  
# Criar cliente S3  
s3_client = boto3.client('s3')  
  
# Listar buckets  
response = s3_client.list_buckets()  
for bucket in response['Buckets']:  
    print(bucket['Name'])  
  
# Usando Resource (alto nível)  
s3_resource = boto3.resource('s3')  
bucket = s3_resource.Bucket('my-bucket')  
for obj in bucket.objects.all():  
    print(obj.key)
```

Gerenciamento de Credenciais no SDK

Ordem de resolução (mesma da CLI - memorize!):

Ordem	Método	Uso
1	Parâmetros no código	NÃO RECOMENDADO (hardcoded)
2	Variáveis de ambiente	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY
3	Shared credentials file	~/.aws/credentials
4	AWS config file	~/.aws/config
5	IAM Role (Instance Profile)	EC2, Lambda, ECS - RECOMENDADO!

MELHOR PRÁTICA: Use IAM Roles para aplicações em EC2/Lambda/ECS!
O SDK obtém credenciais automaticamente via Instance Metadata Service.

Tratamento de Erros e Retry

SDK implementa retry automático com exponential backoff

Tipos de erro que você precisa conhecer:

Erro	Código	Ação
Throttling	429 / ProvisionedThroughputExceeded	Retry com backoff
Service Error	5xx	Retry automático
Client Error	4xx (exceto 429)	NÃO fazer retry - corrigir código
Timeout	RequestTimeout	Aumentar timeout / retry

Exponential Backoff + Jitter:

```
wait_time = base_delay * (2 ^ attempt) + random_jitter  
Exemplo: 100ms, 200ms, 400ms, 800ms... (com variação aleatória)
```

Paginação de Resultados

APIs da AWS retornam resultados paginados para grandes conjuntos de dados

Manual (NextToken):

```
paginator = None
while True:
    if paginator:
        resp = client.list_objects_v2(
            Bucket='bucket',
            ContinuationToken=paginator
        )
    else:
        resp = client.list_objects_v2(
            Bucket='bucket')
    # processar...
    if 'NextToken' not in resp:
        break
    paginator = resp['NextToken']
```

Paginator (RECOMENDADO):

```
paginator = client.get_paginator(
    'list_objects_v2'
)

for page in paginator.paginate(
    Bucket='bucket'
):
    for obj in page['Contents']:
        print(obj['Key'])

# Muito mais simples!
```

DICA: Sempre use Paginators quando disponíveis - código mais limpo e menos bugs!

Pontos-chave para o Exame

1. SDKs e credenciais

- Mesma ordem de precedência da CLI
- IAM Roles para EC2/Lambda = melhor prática

2. Tratamento de erros

- 429/5xx: retry com exponential backoff + jitter
- 4xx (exceto 429): corrigir código, não retry

3. Paginação

- Use Paginators em vez de NextToken manual

LEMBRE-SE: Se pergunta 'erro de throttling' ou '429' > sempre pense exponential backoff com jitter!